

Design Pattern Case Studies

1. Understand the Problem First

Don’t start with “Which pattern should I use?”

Ask:

- What am I trying to achieve? (flexibility, reusability, simplicity, performance)
- Where is the system likely to change in the future?
- Which part of the system should be extensible or replaceable?

2. Categorize the Problem

- Design patterns fall into **three main categories**:
- **Creational Patterns** → Problem with object creation (e.g., too many constructors, need flexible instantiation).
  - *Example*: Use **Factory** if you need to create objects without exposing class details.
- **Structural Patterns** → Problem with class/object composition (e.g., making interfaces compatible, adding features without altering core code).
  - *Example*: Use **Adaptor** when two incompatible interfaces need to work together.
- **Behavioral Patterns** → Problem with communication or responsibility (e.g., controlling request flow, varying behavior).
  - *Example*: Use **Chain of Responsibility** when you want requests to be handled by multiple objects in sequence.

3. Match Pattern to Context (Guidelines)

Singleton → When only **one instance** should exist (e.g., config manager, database connection).

Factory → When object **creation logic is complex** or you want to **hide implementation details**.

Facade → When you want to **simplify a complex subsystem** with one easy-to-use interface.

Decorator → When you need to **add responsibilities dynamically** without changing existing code.

Proxy → When you need to **control access** to an object (e.g., lazy loading, security).

Template → When you have a **fixed workflow with variable steps**.

Adaptor → When two incompatible systems need to **work together**.

Chain of Responsibility → When **multiple handlers may process a request**.

State → When an object's **behavior changes with its internal state**.

4. Consider Change and Extensibility

If **classes might change often** → prefer patterns that allow flexibility (e.g., Strategy, Decorator, Factory).

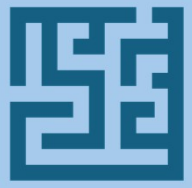
If **you want to isolate dependencies** → use Facade or Adaptor.

If **you want different behaviors without rewriting code** → use State or Template.

5. Balance Trade-offs



Will this pattern **reduce coupling**?



Will it **make code more understandable or more complex**?



Is it **worth the extra abstraction**?

6. Apply “Code Smell → Pattern” Thinking



Often, recurring problems (“code smells”) hint at patterns:



Too many if-else for object creation → **Factory**



Bloated classes with optional features → **Decorator**



Complex subsystem calls scattered everywhere → **Facade**



Object behavior changes via flags/conditions → **State**



Multiple handlers hard-coded in sequence → **Chain of Responsibility**

✅ Summary Guideline



DEFINE THE **PROBLEM & VARIABILITY POINTS**.



CATEGORIZE: CREATION / STRUCTURE / BEHAVIOR.



MATCH THE SITUATION TO THE **INTENT** OF A PATTERN.



CHECK TRADE-OFFS.



APPLY AND TEST IF IT REALLY SOLVES THE PROBLEM.

Case Studies

#	Case	Descripton
1	**Online Food Delivery System** 🍔	Build a system for ordering food online with restaurant integration, payment, and delivery tracking.
2	**Smart Home Automation System** 🏠	Develop a smart home controller for lights, AC, TV, and security.
3	**Online Learning Management System (LMS)** 🎓	Build an LMS where students can access courses, submit assignments, and get feedback.
4	**Banking & ATM System** 🏦	Build a simplified ATM system that allows deposits, withdrawals, and balance checks.
5	**E-Commerce Shopping Cart System** 🛒	Create a shopping cart system with checkout, promotions, and payments.
6	**Travel Booking Platform (Flights, Hotels, Cars)** ✈️🏨🚗	Build a system that integrates multiple booking services for travelers.
7	**Online Video Streaming Platform (like Netflix/YouTube)** 📽️	Understand how multiple design patterns can support content delivery, personalization, and access control in a streaming platform.